```
#include <iostream>
#include <string>
#include <limits>
using namespace std;

// Node representing a song in the playlist
class Node {
public:
    string title;
    string artist;
    Node* prev;
    Node* next;

    Node(string t, string a) {
        title = t;
        artist = a;
        prev = nullptr;
        next = nullptr;
    }
};

class MusicPlaylist {
private:
    Node* head;       // Points to the first song
    Node* current;    // Points to the selected song

public:
    MusicPlaylist() {
        head = nullptr;
        current = nullptr;
    }

    // Add a song at the end
    void addSong(string title, string artist) {
        Node* p = new Node(title, artist);

        if (head == nullptr) {
            head = p;
            current = p;
        } else {
            Node* q = head;  // Traversal pointer

            while (q->next != nullptr) {
                q = q->next;
            }

            q->next = p;
            p->prev = q;
        }
```

```
    cout << "Song added successfully.\n";
}

// Delete a song using its title
void deleteSong(string title) {
    Node* q = head;
    Node* p = nullptr;

    while (q != nullptr && q->title != title) {
        q = q->next;
    }

    if (q == nullptr) {
        cout << "Song not found.\n";
        return;
    }

    p = q;  // p points to the node being deleted

    if (p->prev != nullptr) {
        p->prev->next = p->next;
    } else {
        head = p->next;
    }

    if (p->next != nullptr) {
        p->next->prev = p->prev;
    }

    // Update current if the selected song is deleted
    if (current == p) {
        if (p->next != nullptr) {
            current = p->next;
        } else {
            current = p->prev;
        }
    }

    delete p;

    cout << "Song deleted successfully.\n";
}

// Display songs from first to last
void displayForward() {
    Node* q = head;

    if (q == nullptr) {
        cout << "Playlist is empty.\n";
        return;
```

```
    }

    cout << "\nForward Playlist:\n";

    while (q != nullptr) {
        cout << q->title << " - " << q->artist << "\n";
        q = q->next;
    }
}

// Display songs from last to first
void displayBackward() {
    Node* q = head;

    if (q == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    while (q->next != nullptr) {
        q = q->next;
    }

    cout << "\nReverse Playlist:\n";

    while (q != nullptr) {
        cout << q->title << " - " << q->artist << "\n";
        q = q->prev;
    }
}

// Display the currently selected song
void playCurrent() {
    if (current == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    cout << "Now Playing: " << current->title
         << " - " << current->artist << "\n";
}

// Move to the next song
void playNext() {
    if (current == nullptr) {
        cout << "Playlist is empty.\n";
    } else if (current->next == nullptr) {
        cout << "Already at the last song.\n";
    } else {
        current = current->next;
```

```
        playCurrent();
    }
}

// Move to the previous song
void playPrevious() {
    if (current == nullptr) {
        cout << "Playlist is empty.\n";
    } else if (current->prev == nullptr) {
        cout << "Already at the first song.\n";
    } else {
        current = current->prev;
        playCurrent();
    }
}

// Show the currently selected song
void showCurrent() {
    playCurrent();
}

// Play the complete playlist once
void playCompletePlaylist() {
    Node* q = head;

    if (q == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    cout << "\nPlaying Complete Playlist:\n";

    while (q != nullptr) {
        cout << "Playing: " << q->title
             << " - " << q->artist << "\n";
        q = q->next;
    }

    current = head;
}

// Repeat the complete playlist
void repeatAll(int repetitions) {
    if (head == nullptr) {
        cout << "Playlist is empty.\n";
        return;
    }

    if (repetitions <= 0) {
        cout << "Enter a positive number of repetitions.\n";
```

```
            return;
        }

        for (int i = 1; i <= repetitions; i++) {
            cout << "\nRepetition " << i << ":\n";

            Node* q = head;

            while (q != nullptr) {
                cout << "Playing: " << q->title
                     << " - " << q->artist << "\n";
                q = q->next;
            }
        }

        current = head;
    }

    // Release all allocated nodes
    ~MusicPlaylist() {
        Node* q = head;

        while (q != nullptr) {
            Node* p = q;
            q = q->next;
            delete p;
        }

        head = nullptr;
        current = nullptr;
    }
};

int main() {
    MusicPlaylist playlist;
    int choice;
    string title, artist;
    int repetitions;

    do {
        cout << "\n===== MUSIC PLAYLIST MANAGER =====\n";
        cout << "1. Add Song\n";
        cout << "2. Delete Song\n";
        cout << "3. Display Playlist\n";
        cout << "4. Display Playlist in Reverse\n";
        cout << "5. Play Current Song\n";
        cout << "6. Play Next Song\n";
        cout << "7. Play Previous Song\n";
        cout << "8. Play Complete Playlist\n";
        cout << "9. Repeat All\n";
```

```
cout << "10. Show Current Song\n";
cout << "11. Exit\n";
cout << "Enter your choice: ";

if (!(cin >> choice)) {
    cout << "Invalid input. Please enter a number.\n";
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    continue;
}

cin.ignore(numeric_limits<streamsize>::max(), '\n');

switch (choice) {
case 1:
    cout << "Enter song title: ";
    getline(cin, title);

    cout << "Enter artist name: ";
    getline(cin, artist);

    playlist.addSong(title, artist);
    break;

case 2:
    cout << "Enter song title to delete: ";
    getline(cin, title);
    playlist.deleteSong(title);
    break;

case 3:
    playlist.displayForward();
    break;

case 4:
    playlist.displayBackward();
    break;

case 5:
    playlist.playCurrent();
    break;

case 6:
    playlist.playNext();
    break;

case 7:
    playlist.playPrevious();
    break;
```

```
        case 8:
            playlist.playCompletePlaylist();
            break;

        case 9:
            cout << "Enter number of repetitions: ";

            if (cin >> repetitions) {
                cin.ignore(
                    numeric_limits<streamsize>::max(), '\n'
                );
                playlist.repeatAll(repetitions);
            } else {
                cout << "Invalid repetition count.\n";
                cin.clear();
                cin.ignore(
                    numeric_limits<streamsize>::max(), '\n'
                );
            }
            break;

        case 10:
            playlist.showCurrent();
            break;

        case 11:
            cout << "Exiting Music Playlist Manager.\n";
            break;

        default:
            cout << "Invalid choice. Try again.\n";
        }

    } while (choice != 11);

    return 0;
}
```